

	Ciclo	IF303 - Técnico Superior en Desarrollo de Aplicaciones Web
	Módulo	0483 - Sistemas Informáticos
	Nombre y fecha	
Tipo doc: Examen	Código: EX-IFC303-M0483-2526E1	Página 1 de 9

INSTRUCCIONES: La presentación del examen supone la lectura y aceptación de estos puntos.

En caso de renuncia a la presentación del examen, éste se deberá entregar físicamente con el nombre y la hoja de instrucciones tachada. La renuncia a la presentación implica obtener una calificación de 0 en el mismo.

El incumplimiento de las normas supone la expulsión del examen. La expulsión del examen implica obtener una calificación de 0 en el mismo.

No está permitido: hablar, ceder materiales a los compañeros, el uso de teléfono móvil, el uso de smartwatch, uso de internet, o cualquier otra forma de copia o plagio.

Está permitido utilizar: documentación impresa facilitada para consulta, ordenador personal sin acceso a internet para la parte práctica, **bolígrafo de color azul o negro** para las respuestas al test.

Es necesario obtener cómo **mínimo 5 puntos** en este examen para poder mediar con la nota del trabajo de clase. No es necesario obtener el 50% de la calificación máxima de cada parte, son sumativas.

PARTE A - PREGUNTAS TIPO TEST (3 puntos)

- Solo una respuesta correcta por pregunta
- Cada respuesta correcta: +0,1 puntos
- Respuestas incorrectas: 0 puntos (no restan)
- Sin respuesta: 0 puntos
- Se debe responder con **bolígrafo negro o azul, en MAYÚSCULAS en la cuadrícula que hay a continuación de las instrucciones.**

PARTE B - EJERCICIOS PRÁCTICOS (7 puntos) + EJERCICIO BONUS OPCIONAL (1 punto)

- Se valorará:
 - Funcionamiento correcto del código
 - Código limpio, comentado y con nombres descriptivos
 - Cumplimiento de todos los requisitos especificados
- El ejercicio bonus:
 - Solo se recomienda hacerlo si has terminado los dos ejercicios obligatorios
 - No penaliza si no lo haces o no funciona correctamente
 - La nota máxima del examen es 10, aunque con el bonus puedas llegar a 11 (se redondea a 10)
- Formato de entrega:
 - Hoja del examen con el cuadro de respuestas del test.
 - Carpeta comprimida (.zip) con formato: **Examen1Eval_NombreApellido.zip**
 - Incluir:
 - Carpeta "Ejercicio1" con index.html y script.js

- Carpeta "Ejercicio2" con index.html y script.js
- (Opcional) Carpeta "Ejercicio3" con la misma estructura si haces el bonus

GESTIÓN DEL TIEMPO: Se recomienda la siguiente distribución del tiempo:

- Contenido obligatorio: 150 minutos.
 - Test teórico: 30 minutos. Leer con atención y responder en la cuadrícula.
 - Ejercicios prácticos obligatorios: 100 minutos.
 - Revisión: 20 minutos. Revisar test, probar código, comprobar formato de entrega.
- Ejercicio opcional: 30 minutos. Solo si has terminado lo anterior.

RECOMENDACIONES IMPORTANTES:

1. Lee TODO el examen antes de empezar a escribir código
2. Empieza por el test: es rápido y te ayuda a entrar en contexto
3. Si te atascas en un ejercicio, pasa al siguiente y vuelve después
4. Prueba tu código en el navegador antes de entregarlo
5. Deja tiempo para revisar, errores tontos restan puntos
6. El Ejercicio 3 es BONUS, solo hazlo si te sobra tiempo

RESPUESTAS PREGUNTAS TIPO TEST

[illegible]

	Ciclo	IF303 - Técnico Superior en Desarrollo de Aplicaciones Web
	Módulo	0483 - Sistemas Informáticos
	Nombre y fecha	
Tipo doc: Examen	Código: EX-IFC303-M0483-2526E1	Página 3 de 9

PREGUNTAS TIPO TEST

- ¿Cuál es la diferencia principal entre `let` y `const`?
 - `let` es para números y `const` para texto
 - `const` no permite reasignación, `let` sí
 - `let` tiene scope global, `const` local
 - No hay diferencia
- ¿Qué devuelve `typeof null` en JavaScript?
 - "null"
 - "undefined"
 - "object"
 - "boolean"
- ¿Cuál es el resultado de `"5" + 3` en JavaScript?
 - 8
 - "53"
 - Error
 - NaN
- ¿Qué operador se utiliza para comparar valor Y tipo?
 - ==
 - ===
 - =
 - !=
- ¿Cuál de estas NO es una forma válida de declarar una función?
 - `function suma(a, { return a + b; } }`
 - `const suma = (a, => a + b; }`
 - `const suma = function(a, { return a + b; } }`
 - `function = suma(a, { return a + b; } }`
- ¿Qué método de Math devuelve un número aleatorio entre 0 y 1?
 - Math.rand()
 - Math.random()
 - Math.floor()
 - Math.round()
- ¿Qué devuelve `"Hola Mundo".toUpperCase()`?
 - "hola mundo"
 - "HOLA MUNDO"
 - "Hola Mundo"
 - Error

8. ¿Cómo se obtiene el año actual con el objeto Date?
- `new Date().getYear()`
 - `new Date().year()`
 - `new Date().getFullYear()`
 - `Date.now().year()`
9. ¿Qué método de String divide una cadena en un array?
- `split()`
 - `slice()`
 - `substring()`
 - `divide()`
10. ¿Cómo se redondea 4.7 al entero más cercano?
- `Math.floor(4.7)`
 - `Math.ceil(4.7)`
 - `Math.round(4.7)`
 - `Math.trunc(4.7)`
11. ¿Qué método añade un elemento AL FINAL de un array?
- `shift()`
 - `unshift()`
 - `push()`
 - `pop()`
12. ¿Qué devuelve el método `find()` si NO encuentra ningún elemento?
- `null`
 - `undefined`
 - `-1`
 - `false`
13. ¿Cuál es la diferencia entre `map()` y `forEach()`?
- No hay diferencia
 - `map()` devuelve un nuevo array, `forEach()` no
 - `forEach()` es más rápido
 - `map()` solo funciona con números
14. ¿Qué método se utiliza para filtrar elementos de un array según una condición?
- `find()`
 - `filter()`
 - `reduce()`
 - `some()`
15. ¿Qué hace el método `reduce()`?
- Elimina elementos duplicados
 - Ordena el array
 - Acumula valores en uno solo
 - Filtra elementos
16. ¿Qué propiedad de `window.location` contiene la URL completa?

	Ciclo	IF303 - Técnico Superior en Desarrollo de Aplicaciones Web	
	Módulo	0483 - Sistemas Informáticos	
	Nombre y fecha		
Tipo doc: Examen	Código: EX-IFC303-M0483-2526E1	Página 5 de 9	

- a. window.location.url
- b. window.location.href
- c. window.location.path
- d. window.location.fullUrl

17. ¿Qué método se usa para redirigir a otra página?

- a. window.redirect()
- b. window.location.go()
- c. window.location.href = "url"
- d. window.navigate()

18. ¿Qué propiedad indica el navegador del usuario?

- a. window.browser
- b. navigator.userAgent
- c. window.userAgent
- d. navigator.browser

19. ¿Qué método de **history** vuelve a la página anterior?

- a. history.previous()
- b. history.goBack()
- c. history.back()
- d. history.return()

20. ¿Qué hace **window.innerWidth**?

- a. Cambia el ancho de la ventana
- b. Devuelve el ancho del viewport
- c. Devuelve el ancho de la pantalla
- d. No existe esa propiedad

21. ¿Qué método selecciona UN elemento por su ID?

- a. document.querySelector()
- b. document.getElementById()
- c. document.getElement()
- d. document.selectById()

22. ¿Cuál es la diferencia entre **querySelector()** y **querySelectorAll()**?

- a. No hay diferencia
- b. **querySelector()** devuelve el primero, **querySelectorAll()** todos
- c. **querySelectorAll()** es más rápido
- d. **querySelector()** solo funciona con IDs

23. ¿Qué método crea un nuevo elemento HTML?

- a. document.createNode()
- b. document.newElement()
- c. document.createElement()

d. `document.addElement()`

24. ¿Qué hace `appendChild()`?

- a. Elimina un hijo
- b. Añade un elemento al final de otro
- c. Clona un elemento
- d. Mueve un elemento al principio

25. ¿Cómo se añade un event listener a un botón?

- a. `boton.onClick = function(){}`
- b. `boton.addEventListener("click", function(){} )`
- c. `boton.listen("click", function(){} )`
- d. `boton.on("click", function(){} )`

26. ¿Cuál es la diferencia entre `localStorage` y `sessionStorage`?

- a. `localStorage` persiste, `sessionStorage` se borra al cerrar la pestaña
- b. No hay diferencia
- c. `localStorage` es más rápido
- d. `sessionStorage` tiene más capacidad

27. ¿Qué método se usa para guardar un dato en `localStorage`?

- a. `localStorage.save()`
- b. `localStorage.set()`
- c. `localStorage.setItem()`
- d. `localStorage.add()`

28. ¿Cómo se convierte un objeto JavaScript a JSON?

- a. `JSON.convert()`
- b. `JSON.stringify()`
- c. `JSON.parse()`
- d. `JSON.toJSON()`

29. ¿Qué método se usa para recuperar un dato de `localStorage`?

- a. `localStorage.get()`
- b. `localStorage.retrieve()`
- c. `localStorage.getItem()`
- d. `localStorage.read()`

30. ¿Cómo se convierte un string JSON a objeto JavaScript?

- a. `JSON.toObject()`
- b. `JSON.parse()`
- c. `JSON.stringify()`
- d. `JSON.convert()`

	Ciclo	IF303 - Técnico Superior en Desarrollo de Aplicaciones Web
	Módulo	0483 - Sistemas Informáticos
	Nombre y fecha	
Tipo doc: Examen	Código: EX-IFC303-M0483-2526E1	Página 7 de 9

EJERCICIOS PRÁCTICOS

Instrucciones generales:

- Escribe código limpio y bien comentado
- Los nombres de variables deben ser descriptivos
- Puedes usar tanto funciones tradicionales como arrow functions
- Se valorará la eficiencia y legibilidad del código
- Entrega todos los archivos necesarios (.html, .js)

EJERCICIO 1: Gestor de Tareas con localStorage (4 puntos)

Crea una aplicación web sencilla de gestión de tareas.

1. Requisitos HTML:

- Un input de texto para escribir la tarea
- Un botón "Añadir Tarea"
- Una lista (`<ul>`) donde se mostrarán las tareas
- Cada tarea debe tener un botón "Eliminar"

2. Requisitos JavaScript:

a. Añadir tareas:

- Al hacer clic en "Añadir Tarea", crear un nuevo `<li>` con el texto del input
- Añadir un botón "Eliminar" a cada tarea
- Limpiar el input después de añadir
- Si el input está vacío, mostrar un alert

b. Eliminar tareas:

- Al hacer clic en "Eliminar", eliminar el `<li>` correspondiente del DOM
- Actualizar localStorage después de eliminar

c. Persistencia con localStorage:

- Guardar todas las tareas en localStorage como un array de strings
- Al recargar la página, cargar y mostrar las tareas guardadas
- Usar `JSON.stringify()` para guardar
- Usar `JSON.parse()` para recuperar

d. Código limpio:

- Funciones con nombres descriptivos
- Comentarios explicativos
- Código organizado

EJERCICIO 2 Calculadora de Estadísticas con Arrays (3 puntos)

Crea una página web con una calculadora que procese arrays de números.

1. Requisitos HTML:

- Un textarea donde el usuario puede escribir números separados por comas

- b. Un botón "Calcular Estadísticas"
 - c. Un div donde mostrar los resultados
- 2. Requisitos JavaScript:** Cuando el usuario haga clic en "Calcular Estadísticas", tu código debe:
- a. **Procesar la entrada:**
 - Convertir el texto del textarea en un array de números
 - Ejemplo de entrada: "5, 10, 15, 20" debe convertirse en → [5, 10, 15, 20]
 - b. **Validación:**
 - Si el textarea está vacío, mostrar mensaje de error
 - Si hay texto no numérico, mostrar mensaje de error
 - c. **Calcular estadísticas:**
 - Cantidad total de números
 - Media
 - Mayor valor
 - Menor valor
 - Números pares
 - Números mayores que la media
 - d. **Mostrar resultados:**
 - Mostrar todos los datos calculados en el div de resultados
 - Formato claro y legible

Ejemplo de resultado esperado:

Estadísticas:

- Cantidad de números: 4
- Media: 12.5
- Mayor: 20
- Menor: 5
- Números pares: 10, 20
- Números mayores que la media: 15, 20

EJERCICIO 3: Lista Ordenable con Drag & Drop (1 punto BONUS)

IMPORTANTE: Este ejercicio es **OPCIONAL** y suma hasta 1 punto extra sobre la nota del examen. Solo hazlo si has terminado los ejercicios anteriores y te sobra tiempo.

Crea una lista de tareas que se puedan reordenar arrastrándolas.

- 1. Requisitos HTML:**
 - a. Una lista (`<ul>`) con al menos 5 elementos (`<li>`)
 - b. Cada `<li>` debe tener atributo `draggable="true"`
 - c. Título que indique que se pueden arrastrar
- 2. Requisitos JavaScript:**
 - a. **Evento dragstart:**
 - Capturar qué elemento se está arrastrando
 - Guardar información del elemento que se arrastra
 - b. **Evento dragover:**
 - Permitir soltar elementos sobre otros elementos de la lista

	Ciclo	IF303 - Técnico Superior en Desarrollo de Aplicaciones Web	
	Módulo	0483 - Sistemas Informáticos	
	Nombre y fecha		
Tipo doc: Examen	Código: EX-IFC303-M0483-2526E1	Página 9 de 9	

c. **Evento drop:**

- Reordenar los elementos en la lista
- El elemento arrastrado debe cambiar de posición

3. **Requisitos CSS:**

- a. Cursor que indique que se puede arrastrar
- b. Algún efecto visual al arrastrar (opacidad, borde, etc.)

4. **NO es necesario:**

- a. Persistencia en localStorage
- b. CSS complejo o elaborado
- c. Múltiples columnas (tipo Kanban)